

FLOYD ALGORITHM – SHORTEST PATH (APSP)

(Also called: Floyd's Algorithm, Shortest Path Floyd, or Floyd's APSP DP)

1. Definition

Floyd Algorithm is a dynamic programming-based shortest-path method used to find all-pairs shortest paths (APSP) in a weighted graph that may contain positive or negative weights, but no negative cycles.

2. Purpose

To compute the **minimum cost** (distance) from **every node to every other node**.

3. Works On

- Directed graphs
- Undirected graphs
- Weighted graphs (positive & negative)
- Not valid if negative cycles exist

4. Core Idea (DP Update Rule)

Try to improve the path from $i \rightarrow j$ using an intermediate k :

$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

5. Algorithm

```
// Algo Floyd(G)
// Input: A weighted graph G represented as a distance matrix
dist[V][V]
// Output: Matrix dist[][] containing shortest distances between
every pair of vertices
```

```
1. Initialize dist[i][j] with:
    → weight of edge (i, j), if edge exists
    → 0 if i == j
    → ∞ if no direct edge

2. For k = 1 to V:           // intermediate vertex
    For i = 1 to V:         // source vertex
        For j = 1 to V:     // destination vertex
            if dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

3. Return dist[][]          // final all-pairs shortest
path matrix
```

6. Detecting Negative Cycle

After the algorithm:

If $\text{dist}[i][i] < 0$, → negative cycle exists.

7. Time & Space Complexity

- Time → $O(V^3)$
- Space → $O(V^2)$

8. Applications

- Finding the shortest paths between all pairs
- Distance matrices in networks
- Road/transport planning
- Graph analysis
- Routing tables
- Detecting negative cycles

9. Summary (One Line)

The Floyd Algorithm finds the shortest distance between every pair of vertices using dynamic programming.

WARSHALL ALGORITHM – TRANSITIVE CLOSURE (REACHABILITY)

(Also called: Warshall's Algorithm)

1. Definition

Warshall's Algorithm is a **Dynamic Programming** approach used to compute the **transitive closure** of a directed graph.

It finds whether a **path exists** between any pair of vertices.

2. Purpose

To determine **reachability**:

Can vertex i reach vertex j ?

This algorithm gives a **Boolean reachability matrix**, not distances.

3. Works On

- ✓ Directed graphs
- ✓ Unweighted graphs
- ✓ Graphs stored as adjacency matrix
- ✗ Does not give shortest paths
- ✗ Not meant for weighted graphs

4. Core Idea (DP Update Rule)

If $i \rightarrow k$ and $k \rightarrow j$ are reachable, then $i \rightarrow j$ is reachable:

$$R[i][j] = R[i][j] \text{ OR } (R[i][k] \text{ AND } R[k][j])$$

5. Algorithm

```
// Algo Warshall(G)
// Input: Adjacency matrix R[V][V] of a directed graph G
//         (R[i][j] = 1 if edge i→j exists, else 0)
// Output: Transitive closure matrix R[][] showing reachability
// between all vertex pairs
```

1. Initialize $R[i][j]$ using adjacency matrix:

- $R[i][j] = 1$ if edge (i, j) exists
- $R[i][j] = 0$ otherwise

```

2. For k = 1 to V:           // intermediate vertex
    For i = 1 to V:         // source vertex
        For j = 1 to V:     // destination vertex
            R[i][j] = R[i][j] OR ( R[i][k] AND R[k][j] )

```

```
3. Return R[[]] // final transitive
closure matrix
```